

C A P S T O N E / S M A R T F A C T O R Y V I S I O N Q C

Factory QC

CNN 기반 폐트병 공정 모니터링 시스템

3-Tier(Handler/Service/DAO) 이벤트 기반 아키텍처 · C++17 MainServer · Python asyncio AiServer · MFC Client

기 관
광주인력개발원

팀 원
김혜윤 · 심동주 · 정지훈 · 임완 · 오인효

기 간
12 일 (2026.04.13 → 2026.04.25)

프로젝트 개요

페트병 생산라인의 이물 검출 · 조립 검사를 CNN 기반 비전 AI 로 자동화하고 ,
검사 결과를 실시간으로 DB 저장 · 모니터링하는 Smart Factory 시스템 .

검 사 추 론 지 연

~60ms

AI 추론 end-to-end

A C K 지 연 개 선

<5ms

500ms → <5ms (v0.12)

스 테 이 션

2ea

입고 / 조립

D B 테 이 블

5ea

inspections 외

실시간 비전 검사

Station1 PatchCore(이상탐지) + Station2 YOLO11(객체검출) 이중 모델로 입고 ~ 조립 전 단계 검사 .

이벤트 기반 아키텍처

MainServer EventBus + 워커풀 4 스레드 .
validate 즉시 ACK → 백그라운드 DB/ 이미지 비동기 처리 .

통합 운영 UI

MFC Client 로 실시간 NG 수신 , 검사 pause/resume, 재학습 요청 / 진행률 , 서버 LED 상태 모니터링 .

시스템 구성 — Hub-and-Spoke

MainServer 를 허브로 카메라·AI·DB·Client· 학습 서버가 TCP 로 연결되는 중앙 집중식 구조.



포트 할당	
9000	AiServer → MainServer
9010	MFC → MainServer
9100	MainServer → 학습서버
9101/9102/9201	헬스체크 대상

프로토콜 체계	
100 번째	외부 (MFC ↔ MainServer)
1000 번째	내부 (AI ↔ MainServer)
1100 번째	학습 채널
1200 번째	헬스체크

3 계층 아키텍처	
Handler	프로토콜 라우팅 · 검증
Service	비즈니스 로직 · 트랜잭션
DAO	DB/ 파일 저장
EventBus	워커풀로 비동기 분리

요구사항 분석도

■ AI 모델 학습 · 9개 (AI-001~009)

PART. AI/학습 · AI/운영

PatchCore 학습 (입고)	AI-001	YOLO11 학습 (조립)	AI-002	PatchCore (표면)	AI-003	추론 인터페이스	AI-004	모델 가중치 관리	AI-005	데이터 수집 (입고)	AI-006	데이터 수집 (조립)	AI-007	데이터 증강	AI-008	모델 롤백	AI-009
정상 용기 비지도학습, 백본 ResNet. 224×224		cap / label / liquid_level 3클래스, 입력 640		YOLO ROI crop 후 라벨 표면 이상탐지		Station1/2 Inferencer.infer() 통일		*.ckpt / *.pt 버전 태깅, SCP 배포		페트병 200~300장. 이물/마커/스크래치		캡/라벨 조립 100~200장. 정상/불량		회전, 반전, 밝기/대비, Noise		성능 저하 시 이전 모델 롤백	

■ AI 추론 서버 (Edge PC) · 10개 (EDGE-001~010)

PART. Edge/카메라 · 추론 · 통신 · Arduino · 설정

Pylon 카메라 #1	EDGE-001	Pylon 카메라 #2	EDGE-002	추론 호출 (입고)	EDGE-003	추론 호출 (조립)	EDGE-004	TCP 패킷 송신	EDGE-005	리젝터 제어	EDGE-006	경고기 제어	EDGE-007	설정 관리	EDGE-008	OK 전송 정책	EDGE-009	큐 모니터링	EDGE-010
GigE grab → BGR		GigE grab → BGR		Station1 → PatchCore		Station2 → YOLO+PatchCore		inspection_id + ACK 재전송		시리얼 NG 빨간 LED		RGB LED + I2C LCD		Config.yaml IP/포트		주기/샘플링 전송		지연 감시, 과부하 drop	

■ 응용 서버 (Main Server, C++) · 14개 (MAIN-001~014)

PART. 운용/통신 · 처리 · DB · 저장 · GUI · 헬스체크 · 로그

TCP Listener	MAIN-001	Router	MAIN-002	Station1Handler	MAIN-003	Station2Handler	MAIN-004	Manager	MAIN-005	DbManager	MAIN-006	ImageStorage	MAIN-007
epoll 비동기 수신 · 포트 9000		station 필드 기반 프로토콜 분기		JSON 파싱 → 결과 → defect 정리		JSON 파싱 → defects[] 처리		결과 검증 · 통계 · NG 저장		MariaDB CRUD 전 테이블		./storage/stationN/YYYYMMDD	
GuiNotifier	MAIN-008	HealthChecker	MAIN-009	Protocol 정의	MAIN-010	검사 ID 관리	MAIN-011	ACK 처리	MAIN-012	장애 대응	MAIN-013	통합 로깅	MAIN-014
MFC PostMessage 실시간 푸시		5초 간격 · 3회 무응답 장애		common/Protocol.h 패킷 구조		inspection_id 기반 흐름 추적		DB 저장 후 ACK · 재전송		상태 전파 + 복구 전략		inspection_id 이벤트 로그	

■ MFC 클라이언트 (Admin GUI) · 9개 (MFC-001~009)

PART. MFC/인증 · 템 · 통신 · UI · 권한

사용자 로그인	MFC-001	종합현황 템	MFC-002	① 입고 템	MFC-003
LoginDialog · admin/operator/viewer 권한별 메뉴		OK/NG · 불량률 · 가동률 · 서버 헬스체크		카메라 영상 · OK/NG · 결합 히트맵	
② 조립 템	MFC-004	통계/이력 템	MFC-005	모델관리 템	MFC-006
YOLO 바운딩 박스 + PatchCore 오버레이		불량률 추이 · 파레토 · 기간 필터 · CSV		버전 목록 · 재학습 트리거 · 진행 상태	
NetworkThread	MFC-007	페이지 스위칭	MFC-008	권한 제어	MFC-009
백그라운드 recv → PostMessage UI		ChildView · 템 전환 프레임		관리자/운영자/조화자 기능 접근 제어	

■ Arduino / 하드웨어 · 3개 (HW-001~003)

PART. Arduino · 시리얼 프로토콜

① 입고 리젝터	HW-001	② 조립 경고기	HW-002	시리얼 프로토콜	HW-003
빨간 LED, Station1Rejecter.ino		RGB LED + I2C LCD 16×2 불량 유형		Edge ↔ Arduino USB 시리얼 명령	

요구사항 분석도 — 핵심 정리

기능 요구사항 (FR)

FR-01	실시간 불량 검출 — Station1 PatchCore, Station2 YOLO11, 500ms 이내
FR-02	NG 데이터 수집 — 원본 / 히트맵 / 마스크 3 장, DB inspections INSERT
FR-03	실시간 모니터링 UI — 4 개 탭 (Home/S1/S2/Model), LED 서버 상태 표시
FR-04	사용자 인증 — bcrypt + Salt, Admin/Operator/Viewer 권한
FR-05	모델 재학습 / 배포 — 폴더 업로드 (session_id), TCP 중계 배포
FR-06	통계 / 이력 조회 — STATS_REQ(130), INSPECT_HISTORY_REQ(114)
FR-07	검사 원격 제어 — Pause/Resume, INSPECT_CONTROL(160/161)
FR-08	Arduino 연동 — 발광 / 표지 LED, 시리얼 COM3/COM4
FR-09	헬스체크 — 5 초 주기, 동적 주기, server_type 태깅

제약사항 (Constraints)

C-01	네트워크 — 내부망 전용 (10.x/192.168.x), TCP Only, JSON + Binary 프레임
C-02	크기 제한 — JSON ≤ 1MB, 이미지 ≤ 50MB, 모델 ≤ 500MB
C-03	동시 접속 — AI 서버 10 개, GUI 세션 20 개
C-04	플랫폼 — MainServer: Linux / C++17, AiServer: Python 3.10+, Client: Windows MFC
C-05	하드웨어 — Basler Pylon 카메라, NVIDIA GPU(학습), Arduino Uno
C-06	DB — MariaDB 단일, 5 개 테이블, ConnectionPool = 4

비기능 요구사항 (NFR)

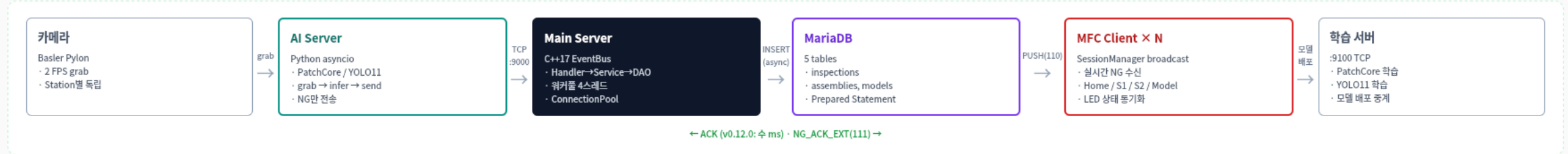
NFR-01	저지연 ACK — v0.12.0 : 500ms → 수 ms 로 개선
NFR-02	TCP Keepalive — 60s idle → 10s × 3 probe, 90 초 내 감지
NFR-03	EventBus 병렬 — 워커풀 4 스레드, asyncio.Queue 백프레서
NFR-04	Defense in Depth — Prepared Statement, IP 화이트리스트, JSON 이스케이프
NFR-05	자동 복구 — mysql_ping 재연결, Atomic File Write
NFR-06	통합 로깅 — logs/YYYY-MM-DD.log, 자정 로테이션
NFR-07	3 계층 아키텍처 유지보수성 — Handler → Service → DAO, SRP
NFR-08	단일 Config — config.json 통합, C++/Python/MFC 공유
NFR-09	한글 UI — Tahoma 9pt, 4 개 탭, LED tri-state

데이터 요구사항 (Data Model)

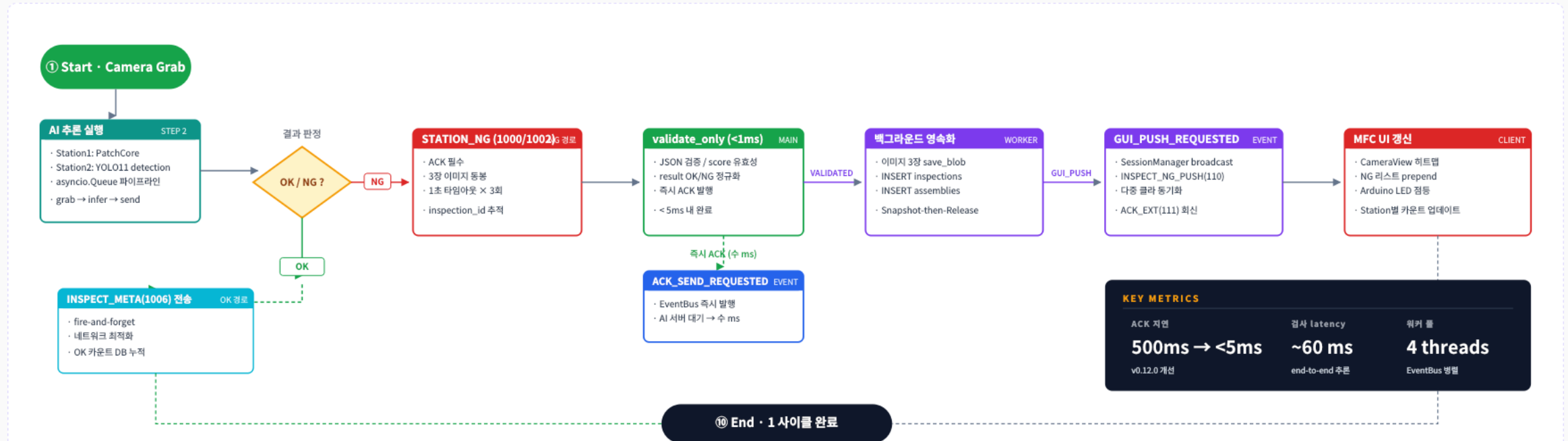
users	id, employee_id, username, password_hash(bcrypt), role
models	id, station_id, model_type, version, file_path, accuracy
bottles	id, product_code, batch_id, created_at
inspections	id, inspection_id(uuid), station_id, result, score, defect_type
assemblies	id, inspection_id FK, cap_ok, label_ok, fill_ok, yolo_detections
Storage(File)	./storage/images/, ./storage/models/, ./logs/

시스템 순서도

■ ① 시스템 구성 (Hub-and-Spoke)

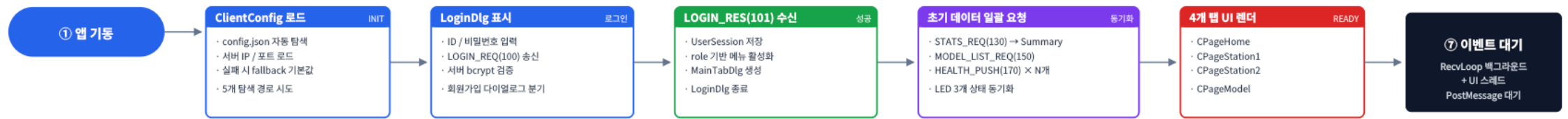


■ ② NG 검사 이벤트 순서 (v0.12.0 비동기 분리)

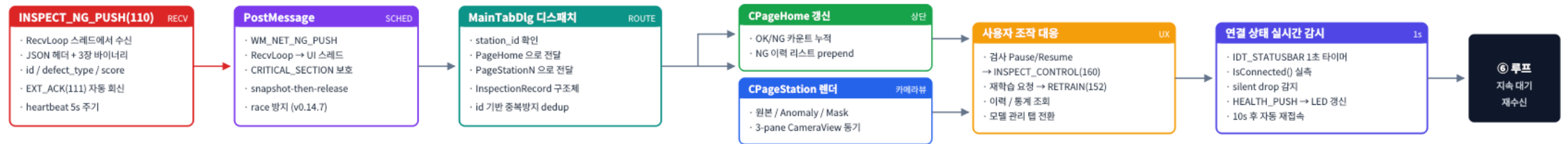


MFC 클라이언트 순서도

① MFC 클라이언트 기동 · 인증 · 초기화



② 실시간 NG 수신 · UI 갱신 흐름



NG 검사 1 사이클 — 단계별 설명

- 01 **Camera Grab**
Basler Pylon 2 FPS · LatestImageOnly 전략
- 02 **AI 추론 실행**
Station1 PatchCore anomaly / Station2 YOLO11 detection
- 03 **결과 판정 (OK/NG)**
OK → INSPECT_META(1006) fire-and-forget
NG → STATION_NG(1000/1002)
- 04 **validate_only**
JSON 검증 + score 유효성 + OK/NG 정규화 (<1ms)
- 05 **즉시 ACK 발행**
AI 서버 대기 500ms → 수 ms 로 단축 (v0.12.0 핵심 개선)
- 06 **백그라운드 영속화**
EventBus 워커풀 4 스레드 · 이미지 3 장 save_blob · DB INSERT
- 07 **MFC UI 갱신**
CameraView 히트맵 · NG 리스트 prepend · Arduino LED 점등

KEY METRICS

ACK 지연

<5 ms

500ms → v0.12.0 개선

검사 latency

~60 ms

end-to-end 추론

NG 재전송

1s × 3

실패 시 최대 3 회

워커 풀

4 threads

EventBus 병렬 처리

핵심 기술

MainServer

Linux · C++17

- TcpListener (포트 9000 / 9010)
- EventBus 워커풀 4 스레드 (큐 상한 10000)
- 3 계층 분리 : Handler → Service → DAO
- ConnectionPool × 4 (mysql_ping 재연결)
- TCP Keepalive 60s idle / 10s × 3
- Prepared Statement + IP 화이트리스트
- bcrypt 비밀번호 해시
- JSON 스택 파서 (자체 구현)

AiServer

Python 3.10+ · asyncio

- Basler Pylon SDK (pypylon) 2 FPS
- asyncio.Queue 파이프라인 (grab → infer → send)
- Station1 PatchCore (anomalib)
- Station2 YOLO11 (ultralytics)
- loop.run_in_executor (GPU 분리)
- ACK 기반 재전송 (3s × 3 회)
- 원자적 모델 저장 (tmp → fsync → rename)
- 학습 서버 : MODEL_RELOAD 지수 백오프

MFC Client

Windows · Visual Studio 2022

- NetworkClient (비동기 수신 스레드)
- 4 개 탭 UI : Home / S1 / S2 / Model
- CameraView 커스텀 뷰 + GDI 렌더
- silent drop 감지 (1s 타이머)
- 자동 재접속 (10s)
- UTF-8 한글 정상 처리
- 서버 LED tri-state
- 회원가입 / 로그인 (서버 DB 인증)

MariaDB

inspections · assemblies · models · bottles · users
(ConnectionPool × 4)

학습 서버

Station 별 PatchCore / YOLO 전이학습 , TRAIN_PROGRESS
스트리밍

config.json

IP · 포트 · 경로 · DB · 하이퍼파라미터 — 3 개 서버 단일 소스

AI 추론 결과 — 예상안 vs 시행후

PatchCore · YOLO11 두 모델 각각 실제 환경에서 추론 테스트를 진행한 결과와 회고.

Station 1

PatchCore

입고 검사 · 이상탐지 (Anomaly Detection)

예상안 (BEFORE)

빈 페트병 외관 이상탐지로 정상 / 불량을 명확히 분리. 학습된 임계값이 한번에 현장에 적용 가능할 것.

시행후 (AFTER)

- 정상 OK 점수 0.15 ~ 0.44 구간
- 불량 NG 점수 0.50 ~ 1.00 구간으로 분리 성공
- 파이리 1.00 · 하얀종이뭉치 0.50~0.97
- 보드마카 0.70 · 스크래치 0.48~0.69 검출

이슈 정상 이상점수가 ~0.46 인 데 비해 스크래치가 0.48~0.59 → 두 분포가 근접
해결 임계치 0.47 로 재조정 제안. 근본 원인은 카메라 - 페트병 거리 / 각도 통제 부족 — 촬영환경 안정화 필요

Station 2

YOLO11

조립 검사 · 객체탐지 (Object Detection)

예상안 (BEFORE)

cap / label / liquid_level 3 개 클래스의 존재여부 + 위치상태까지 IoU 기반 정확 검출.

시행후 (AFTER)

- 존재여부 (캡없음 / 라벨없음) → 확실히 검출
- 위치상태 (기울어짐 · 높낮이) → 하나도 못잡음
- 위치 판정 시 정상 병도 전부 NG
- 원인: 조립공정도 촬영환경 변화 영향 동일

이슈 위치 IoU 판정 로직이 조명 / 각도 변화에 과민 반응 — 실환경에서 False-NG 폭증
해결 v0.15.4: 위치상태 판정 로직 제거. 캡 / 라벨 / 충전 존재여부만으로 OK/NG 판정하도록 단순화

THANK YOU